Constraint-Aware E-Graph Rewriting for Hardware Performance Optimization

Samuel Coward, Theo Drane, and George A. Constantinides, *Senior Member, IEEE,*

***Abstract*—Data dependent constraints commonly occur across hardware and software, often in the form of code branches or input constraints. Expert designers exploit these constraints to realize new optimization opportunities. Numerical hardware designers exploit this aggressively, as if a particular module input value can never be seen, then there is no need to dedicate any circuit area to handling that input value. Floating-point hardware designers have gone further, specifically introducing carefully constructed case-splits to exploit under-utilized critical paths. To automate constraint-aware optimization, we developed a theoretical framework, based on the e-graph data structure, that localizes constraint reasoning tasks and makes it simple to realize optimizations exploiting the underlying control structures. The theory introduced here, provides an approach to encode multiple equivalence relations within a single e-graph.**

To demonstrate the value of the theoretical developments, we extend an existing RTL optimization tool, ROVER, allowing it to exploit constraints present in the RTL itself. We combine this new constraint-awareness with a new RTL value range analysis, allowing ROVER to understand what values each intermediate signal can take. We further add to ROVER by developing a model for circuit delay, allowing ROVER to explore the trade-offs between performance and circuit area. With these latest developments, ROVER is capable of fully-automatically discovering known floating-point architectures from the computer arithmetic literature. The designs generated by constraint-aware ROVER are, on average, 30% faster and 1% smaller than those generated by state-of-the-art EDA tools.

***Index Terms*—hardware optimization, design automation, datapath design, computer arithmetic**

I. Introduction

CONSTRAINT-AWARE optimization provides an opportunity to exploit common control structures such as code branches. Such control structures are commonplace in hardware and software, but automating constraint-aware optimization is challenging, because one must infer the impact of a constraint on an arbitrary code segment. This involves automatically identifying variable correlations and performing a reasoning step.

We consider constraint-aware optimization to be a combination of domain refinement and sub-domain equivalence. This perspective allows us to bridge the gap between constraint-aware optimization and a program optimization technique, equality saturation [1], [2], which has gained popularity in recent years [3]–[5]. Equality saturation is an optimization technique that leverages an e(quivalence)-graph data structure. As it's name would suggest, e-graphs are used to reason over equivalence relations, which will provide a barrier to exploring constraint-aware optimization.

Taking inspiration from the program analysis community [6], we propose a general solution to encode sub-domain equivalences in an e-graph via the introduction of *special* `ASSUME` operators that localize constraints. By defining rewrites over `ASSUME` operators we can encode constraint-aware optimizations within an e-graph setting. This is valuable because we can then combine these optimizations with existing work to enhance equality saturation based optimizers. Furthermore, we can exploit the e-graph to efficiently explore a range of constraint-aware optimizations, deferring the benefit analysis to a later implementation selection stage.

To demonstrate the value of constraint-aware optimization, we apply the theory to register transfer level (RTL) optimization for datapath designs. Constrained sub-circuits occur naturally when a designer inserts a conditional mux, joining two branches. Such conditions are manually exploited by expert engineers as they often represent additional optimization opportunities. In fact, designers may even specifically insert case-splitting muxes to facilitate further performance optimization. State-of-the-art floating point adder architectures, utilize a carefully designed case-split [7], [8].

In this work, we show that these manual optimizations can be automated and generalized by our constraint-aware e-graph rewriting framework. We combine constraint-aware rewriting with a bit-accurate value range analysis. We develop a tool, ROVER, capable of optimizing an RTL design to maximize performance, but also capable of balancing the area-delay trade-off generating a Pareto frontier of implementations.

The primary contributions of this work are:

- a general purpose encoding of multiple equivalence relations and constraint-aware optimization via e-graph rewriting,
- an accurate value range analysis for bitvector arithmetic in RTL designs,
- a theoretical model of circuit delay and an integer linear programming method to balance the area-delay trade-off and generate a Pareto frontier of designs,
- detailed case-studies and benchmark results demonstrating the automation of sophisticated RTL optimization.

We presented an initial study on encoding constraint-aware optimization via e-graph rewriting at the 60th Design Automation Conference [9] as a preliminary version of this work. Here we extend the conference paper with contributions to general purpose e-graph theory and application specific RTL optimizations. Firstly, we provide a complete semantics of

S.Coward and T.Drane are with Intel, Numerical Hardware Group, USA. E-mail: samuel.coward@intel.com

G.A.Constantinides and S.Coward are with Imperial College London, Department of Electronic and Electrical Engineering, London, UK

Manuscript received July 16, 2024

the general purpose `ASSUME` operator, which we hope will simplify the adoption of the approach in a range of e-graph applications. To improve performance we introduce a new analysis technique that limits the exploration of unprofitable architectures. To RTL optimization, we contribute an extension to ROVER's interval arithmetic based analysis to signed bitvector arithmetic. We also develop a method to extract a Pareto frontier of designs from a single e-graph, balancing the area-delay trade-off.

The paper is organized as follows. We first provide the necessary background on e-graphs and datapath circuit optimization in Section II. We then describe the key contribution of this work in Section III, the `ASSUME` operator, and how it enables the expression of sub-domain equivalence relations in the e-graph data structure. In Section IV, we describe an application to datapath circuit optimization. Lastly, in Section V, we demonstrate the impact on circuit performance and area using this technique.

II. Background

We provide a brief background on three key concepts that underpin both the theoretical contributions and the practical application built on this theory. Firstly, in Section II-A, we introduce the e-graph data structure, which we use for design space exploration. Secondly, in Section II-B, we discuss previous work on program analysis, with a particular focus on abstract interpretation. Lastly, in Section II-C, we will introduce datapath design concepts that will be utilized in our RTL optimization application.

A. E-Graphs

The e-graph is a data structure widely used to compactly represent sets of equivalent expressions [10]. Via a technique known as equality saturation [1], [2], these sets of equivalent expressions are added to or merged to discover new equivalent expressions. Equivalent expressions are clustered together into equivalence classes (e-classes) with nodes in the graph representing variables, constants or functions. Edges connect function nodes to their operand e-classes, meaning that a small number of nodes can represent exponentially many more expressions. Figure 1 contains an e-graph, demonstrating how multiple equivalent expressions can be represented.

Equality saturation is based on the theory of uninterpreted functions, where operators only gain meaning via rewrites and analyses defined on them. Rewrites, $l \rightarrow r$ define equivalent expressions such that when l is matched in an e-graph, the expression r gets added to the same e-class. For example, for any expression x, $x + x \rightarrow 2 \times x$ encodes the equivalence $x + x \cong 2 \times x$ in the e-graph. Equality saturation depends on a user defined set of rewrites. Opportunities to apply a rewrite are detected via e-matching, which searches the e-graph for expressions matching the left-hand side patterns [11], [12]. Equality saturation applies rewrites constructively, that is, when a rewrite $l \rightarrow r$ is applied, the left-hand side is not destroyed and remains in the e-class. As a result the e-graph grows monotonically and application of one rewrite does not remove alternative rewriting opportunities. This helps to mitigate the traditional compiler ordering problem. Equality saturation may reach a fixed point called saturation, where no further rewrite applications add information to the e-graph.

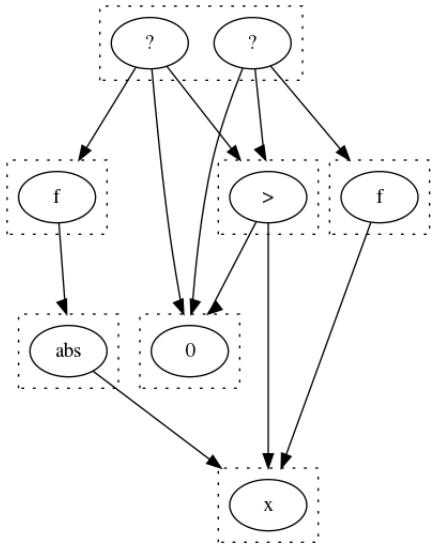


Fig. 1: An e-graph representing two equivalent expressions, `x>0?f(abs(x)):0` ≅ `x>0?f(x):0`. Dashed boxes represent e-classes of equivalent expressions.

A recent wave of e-graph research has been fueled by the release of a general purpose e-graph library, `egg` [2], which has opened new avenues thanks to performance enhancements. Two additional `egg` innovations will be of key value to this work. First, an e-class analysis feature allows users to attach meta-data to an e-class, facilitating program analysis on an e-graph [13]. Second, proof production allows users to extract a rewrite sequence mapping one expression to an equivalent expression in the e-graph [14], valuable for proof decomposition and translation validation.

With the introduction of equality saturation, e-graphs have moved from traditional applications such as SMT solvers [15], and on to optimization challenges. For example, the `egg` library is a core component of the Herbie tool that automates numerical stability analysis [16]. Equality saturation has also gained traction in the electronic design automation (EDA) community, with the authors of [17] advocating its broader application. `egg` has been used to explore alternative multiplier implementations [4], [18] and resolve phase-ordering challenges in high-level synthesis [19]. In previous work, we tackled datapath circuit area optimization using `egg` [3], [20], developing a tool called ROVER. To demonstrate the benefits of the theoretical contributions described in Section III, we extend this circuit optimization framework, adding support for constraint-aware optimization that significantly expands ROVER's optimization capabilities.

A fundamental limitation of the e-graph is that all rewrites define equivalence with respect to the same equivalence relation. This will be problematic in the context of constraint-aware optimization. Consider the following expression con-

taining an arbitrary function `f`.

$$\texttt{(x>0 ? f(abs(x)) : 0)} \cong \texttt{(x>0 ? f(x) : 0)} \quad (1)$$

Whilst these two expressions are clearly equivalent, it is difficult to express via local rewrites since, in general, the rewrite $\texttt{abs(x)} \rightarrow \texttt{x}$ is not valid. However, the constraint-aware rewrite $\texttt{x>0} \Rightarrow \texttt{abs(x)} \rightarrow \texttt{x}$ is always valid. An e-graph representing these two expressions is shown in Figure 1. To resolve this conflict, one prior work proposes "Colored E-Graphs" [21]. In a colored e-graph, different equivalence relations (each represented by a different color) are layered on the e-graph, creating a hierarchy. The colored e-graph approach adds significant complexity and requires modification to the underlying equality saturation algorithms. In contrast, the `ASSUME` operators, proposed in this work, require no modification to the equality saturation algorithms and can similarly express different equivalence relations.

B. Program Analysis

Constraint-awareness not only provides benefits for program optimization, but also yields refinement advantages for program analysis techniques. Compilers use program analysis to check the validity of a range of optimizations, for example dead code elimination [22]. A widely used theory underpinning program analysis is abstract interpretation, which over-approximates program properties using an abstract domain [23]. A simple, but relatively weak, interpretation is interval arithmetic, approximating numerical variables by their input ranges, and operators by their natural interval extension.

Abstract interpretation is used in static program analysis [24], where the program is analyzed without being executed. A general objective of static analysis tools is to efficiently compute precise abstractions of program properties. Unfortunately, cheap to compute interpretations typically over-approximate. To counter this over-approximation, existing tools have incorporated term rewriting to discover "analysis-friendly" program representations [25]. Our work benefits from the fact that alternative program representations yield distinct abstract interpretations [13]. For example, consider two integer variables $a \in [-2, 2]$ and $b \in [-1, 1]$, an interval interpretation of equivalent expressions $a \times b - b$ and $(a-1) \times b$ yields $[-4, 4]$ and $[-3, 1]$, respectively.

Capturing constraints in an abstract interpretation is typically more computationally expensive and involves analysing relational domains. Prior work has combined constraint-awareness with interval arithmetic [26], which we shall also combine in our application, described in Section IV. This work takes inspiration from [6], where the authors introduce special `ASSUME` operators, that couple sub-programs with the constraints (introduced by control structures) which can be assumed upon evaluation. This coupling facilitates abstraction refinement, yielding a more precise abstraction.

In [13], we demonstrated the advantages of combining abstract interpretation with e-graph rewriting. Using an e-class analysis, an abstract element can be attached to each e-class. We showed in [13], that a sound abstraction for one expression in an e-class is a sound abstraction of the entire class. For each node in an e-class, we generate an associated abstract element and take the greatest lower bound (meet) across all of these elements in the e-class, yielding a sound and refined abstraction. This prior work also briefly highlighted the connection to `ASSUME` operators and abstraction refinement.

C. Datapath Synthesis

In Section IV, we demonstrate how the theoretical contributions described in Section III can be applied, in practice, to an RTL optimization tool. In particular, we target datapath synthesis, the process of generating gate level netlists from higher-level arithmetic circuit designs expressed in RTL [27]. Datapath synthesis engines are utilized in front-end logic synthesis passes, performing arithmetic optimizations and mapping word-level operators into gate-level implementations. These tools represent the state of the art in datapath optimization and will be our comparison point throughout.

Commercial logic synthesis tools detect opportunities to exploit redundant number representations by clustering arithmetic operators together [27], [28]. To the best of our knowledge, commercial synthesis tools do not exploit data dependent constraints in the RTL, for example, those introduced via muxes, to optimize for area or delay. Power optimization tools such as Siemens PowerPro [29] do exploit the mux structure in the RTL, as branches introduce redundant computations which can be disabled to save power.

The optimization tool we extend, ROVER, is an e-graph based RTL rewriting tool, which has already been shown to produce circuit area reductions [3]. This extension incorporates constraint-aware optimization and targets critical path delay reduction. Prior work also deployed local dataflow transformations to facilitate the use of carry-save representation when presented with consecutive arithmetic operations [30]. A general purpose RTL rewriting framework, Voss II, has been developed by Carl Seger and collaborators [31], [32], where the emphasis is on verified interactive exploration. In FPGA design, rewriting has been applied to the design of large bitwidth multipliers [4]. The optimizations performed by constraint-aware ROVER go beyond prior rewriting approaches, and capture deeper optimizations that are typically hand-coded by datapath experts.

III. Localizing Constraint-Aware Optimization

An e-graph represents a set of expressions, the variables of which we can take to range over a domain $\mathcal{D}$. Let *Expr* denote the set of all representable expressions of n variables. Imposing a canonical ordering on the set of variables, the space of permissible inputs is constrained to a defined subset $J \subseteq \mathcal{D}^n$. This restriction can encode constraints on each input variable, for example type restrictions. We define a concrete semantics of expressions

$$[\![\cdot]\!]_\cdot \in \mathit{Expr} \rightarrow J \rightarrow \mathcal{D}. \quad (2)$$

Under these semantics $[\![e]\!]_\rho$ denotes the interpretation of expression e under execution environment $\rho \in J$. For example, let $J = \mathbb{Z}^2$ and $\rho = \{a \rightarrow 2, b \rightarrow 3\}$, then $[\![a + b]\!]_\rho = 5$.

A. Sub-Domain Equivalence

An e-graph represents a single congruence relation.

Definition 1 (Congruence)**.** Two expressions e_a and e_b are congruent, $e_a \cong e_b$, iff $[\![e_a]\!] = [\![e_b]\!]$.

This notion of congruence requires equality across the entire domain of possible inputs J. However, the presence of input constraints and code branches means that many sub-expressions are only ever evaluated (or are only utilized when evaluated) on a restricted subset of J, constituting a domain refinement. Consider again (1) and let $\mathtt{x} \in \mathbb{Z}$. The true branch imposes a domain refinement, such that on this branch $\mathtt{x} \in \mathbb{N} \setminus \{0\}$. As a result, we can relax the requirement on such a strict notion of equivalence for such sub-expressions. To specify equivalence on a restricted domain, consider $J' \subseteq J$ and define a sub-domain congruence relation.

Definition 2 (Sub-Domain Congruence)**.** Given $J' \subseteq J$, two expressions e_a and e_b are congruent in the sub-domain defined by J', $e_a \cong_{J'} e_b$, iff $[\![e_a]\!]_\rho = [\![e_b]\!]_\rho$ for all $\rho \in J'$.

Note that, for $J' \subset J$, $e_a \cong_{J'} e_b \not\Rightarrow e_a \cong e_b$. Such sub-domain congruence relations form a lattice, where the top element $\top$ is represented by $\cong$ and $\bot$ is represented by $\cong_\emptyset$, as all expressions are congruent on the empty set. The meet and join of this lattice are defined as follows.

$$\cong_{J_1} \sqcap \cong_{J_2} := \cong_{J_1 \cap J_2} \tag{3}$$

$$\cong_{J_1} \sqcup \cong_{J_2} := \cong_{J_1 \cup J_2} \tag{4}$$

In practice, sub-domain equivalence relations become relevant in the presence of conditional branches and input constraints. Let *BoolExpr* $\subseteq$ *Expr* denote the subset of expressions that under evaluation map to $\{\mathtt{True}, \mathtt{False}\}$. Given $\mathtt{c} \in$ *BoolExpr*, let $[\![\mathtt{c}]\!]^T = \{\rho \in J \mid [\![\mathtt{c}]\!]_\rho = \mathtt{True}\}$, namely the sub-domain on which $\mathtt{c}$ evaluates to true. Vice versa, let $[\![\mathtt{c}]\!]^F$ denote the sub-domain on which $\mathtt{c}$ evaluates to false. These sets correspond to congruence relations satisfying:

$$\cong_{[\![\mathtt{c}]\!]^T} \sqcup \cong_{[\![\mathtt{c}]\!]^F} := \cong . \tag{5}$$

To implement constraint-aware optimizations, we utilize the following observation:

$$e_a \cong_{[\![\mathtt{c}]\!]^T} e_b \iff c \vDash e_a \cong e_b. \tag{6}$$

We demonstrated an example of a valuable sub-domain equivalence in (1).

B. Sub-Domain Equivalence in an E-Graph

All e-classes within an e-graph are equivalence classes with respect to a single concrete notion of congruence, $\cong$, therefore defining any rewrite $e_a \to e_b \Rightarrow e_a \cong e_b$. To represent sub-domain congruence relations, we introduce a special ASSUME operator. The ASSUME operator takes two operands, an *Expr* to evaluate and a *BoolExpr* that defines the sub-domain of interest. To the input domain $\mathcal{D}$, we append an additional *special* element, forming a new domain $\mathcal{D}' = \mathcal{D} \cup \{\bot\}$. Using this element, $\bot$, we define a semantics of ASSUME, such that only when the condition holds are we interested in the result of the expression, and everywhere else, two ASSUME operators with identical conditions will match.

$$[\![\mathtt{ASSUME}(\mathtt{x}, \mathtt{c})]\!]_\rho = \begin{cases} [\![\mathtt{x}]\!]_\rho & \text{if } [\![\mathtt{c}]\!]_\rho, \\ \bot & \text{else.} \end{cases} \tag{7}$$

Then, for any n-ary function f, if any $[\![\mathtt{x}_i]\!] = \bot$:

$$[\![f(\mathtt{x}_1, \ldots, \mathtt{x}_n)]\!] = \bot. \tag{8}$$

It follows from (7) that:

$$\mathtt{x} \cong_{[\![\mathtt{c}]\!]^T} \mathtt{y} \Leftrightarrow \mathtt{ASSUME}(\mathtt{x}, \mathtt{c}) \cong \mathtt{ASSUME}(\mathtt{y}, \mathtt{c}). \tag{9}$$

Crucially, on inputs where $[\![\mathtt{c}]\!]$ does not evaluate to true, the equivalence in (9) is vacuously satisfied. Returning to example (1) from Section II-A. We can encode the desired equivalence as

$$\mathtt{ASSUME}(\mathtt{abs}(\mathtt{x}), \mathtt{x} > 0) \cong \mathtt{ASSUME}(\mathtt{x}, \mathtt{x} > 0). \tag{10}$$

Table I describes a set of rewrites, encoding the creation, propagation and refinement of ASSUME operators. In this work, ASSUME operators are derived purely from code branches. The "Propagate" rewrite allows ROVER to localize reasoning tasks, since the constrained sub-expression may appear at any point within the code branch. Consider the following example, where f and g represent arbitrary functions:

$$\begin{aligned} \mathtt{ASSUME}(f(g(\mathtt{x})), \mathtt{x} == 1) &\to f(\mathtt{ASSUME}(g(\mathtt{x}), \mathtt{x} == 1)) \\ &\to f(g(\mathtt{ASSUME}(\mathtt{x}, \mathtt{x} == 1))). \end{aligned}$$

The propagation rewrite encodes the search to locate the constrained sub-expression.

Propagating an ASSUME operator through an e-graph duplicates the sub-e-graph. In circumstances where there is no constrained sub-expression in the branch, this adds useless nodes to the e-graph. To minimize sub-e-graph duplication, we maintain a live e-class analysis on the e-graph. In each e-class, we store a list of e-classes reachable via any path starting from that e-class. More formally, let E denote the set of e-graph edges and $L(x)$ return the live e-classes associated with e-class x. Then L is calculated as follows.

$$L(x) = \{x\} \cup Y, \text{ where } Y = \bigcup_{\substack{n \in x \\ (n,y) \in E}} L(y)$$

The "Create" rewrites are conditionally applied when the live-variables of the constraint e-class and evaluated expression have a non-empty intersection. More formally, for an expression over e-classes a, b and c,

$$\begin{aligned} &\text{if } L(a) \cap L(b) \neq \emptyset \\ &\quad a \ ?\ b\ :\ c \to a \ ?\ \mathtt{ASSUME}(b, a)\ :\ c. \end{aligned}$$

In hardware design, control signals, such as resets, offer no datapath optimization opportunities since they are typically not correlated with the data signals. The "Propagate" rewrite is also conditionally applied based on the live-class analysis. Such a conditional ASSUME creation and propagation approach only introduces the overhead of e-graph duplication when it has the potential to be profitable. Empirical evidence is provided in Section V.

TABLE I: ASSUME node rewrites, describing creation, propagation and refinements. op represents any operator defined in the expression language. The two argument op extends naturally to arbitrary arity. Boolean negation is represented by $\sim$.

Name	Left-Hand Side	Right-Hand Side
Create	$a\,?\,b : c$ $a\,?\,b : c$	$a\,?\,\texttt{ASSUME}(b,\ a) : c$ $a\,?\,b : \texttt{ASSUME}(c,\ \sim a)$
Propagate	$\texttt{ASSUME}((a\ \texttt{op}\ b),\ c)$	$\texttt{ASSUME}(a,\ c)\ \texttt{op}\ \texttt{ASSUME}(b,\ c)$
Combine	$\texttt{ASSUME}(\texttt{ASSUME}(a, b),\ c)$	$\texttt{ASSUME}(a,\ b \,\&\, c)$
Refine	$\texttt{ASSUME}(a, \texttt{False})$ $\texttt{ASSUME}((a\,?\,b : c),\ a)$ $\texttt{ASSUME}((a\,?\,b : c),\ \sim a)$	$\texttt{ASSUME}(0, \texttt{False})$ $\texttt{ASSUME}(b,\ a)$ $\texttt{ASSUME}(c,\ \sim a)$

In the presence of nested mux structures, constraints on a sub-expression can be accumulated via the "Combine" rewrite. The combination of constraints may yield conflicts, which, after applying Boolean rewrites, produces an $\texttt{ASSUME}(a, \texttt{False})$ expression. This expression corresponds to dead-code, which we can eliminate by rewriting to something trivial e.g. $\texttt{ASSUME}(0, \texttt{False})$. This is shown as the first "Refine" rewrite in Table I. The last two "Refine" rewrites represent other dead-code elimination optimizations.

The general rewrites in Table I can be extended to encompass application specific optimizations, for example (10). To illustrate the advantage of combining constraint-aware optimization with equality saturation, consider this toy example.

$$\texttt{x} == \texttt{y}\,?\,\texttt{x} + \texttt{y} : \texttt{z} \tag{11}$$

Whilst a simplified expression is obvious to a human, this may prove challenging for a traditional optimizer. However, the simplification can be realized via constraint-aware optimization and equality saturation.

```
    x == y  ? x + y                          : z →
  x-y == 0  ? x - y + 2y                     : z →
  x-y == 0  ? ASSUME(x - y + 2y, x-y==0)     : z →
  x-y == 0  ? ASSUME(0 + 2y, x-y==0)         : z →
  x-y == 0  ? ASSUME(2y, x-y==0)             : z
```

The example, utilizes a total of five rewrites, of which the first three illustrated can be applied in parallel.

C. Program Analysis Refinement

We have demonstrated how ASSUME operators can encode sub-domain equivalences via specifically defined rewrites. However, the origins of ASSUME operators lie in abstract interpretation [6], where they modify an abstract element assuming the associated constraint holds. This hints at a connection to the e-class analysis described in Section II-A. For a given expression, e, we use abstract interpretation to associate an abstract element $\mathcal{A}[\![e]\!]$ with that expression, and similarly, given an e-class c, we associate an abstract element $\mathcal{A}[\![c]\!]$ with each e-class. In an e-graph representation of ASSUME, its children are in fact e-classes which, via an e-class analysis, have an associated abstract element. Therefore, we can naturally transfer the abstract interpretation definition of ASSUME and apply it to an e-graph. Furthermore, by exchanging both the expression under evaluation and assumed condition by e-classes of expressions, we simplify complex equational reasoning to a syntactic check.

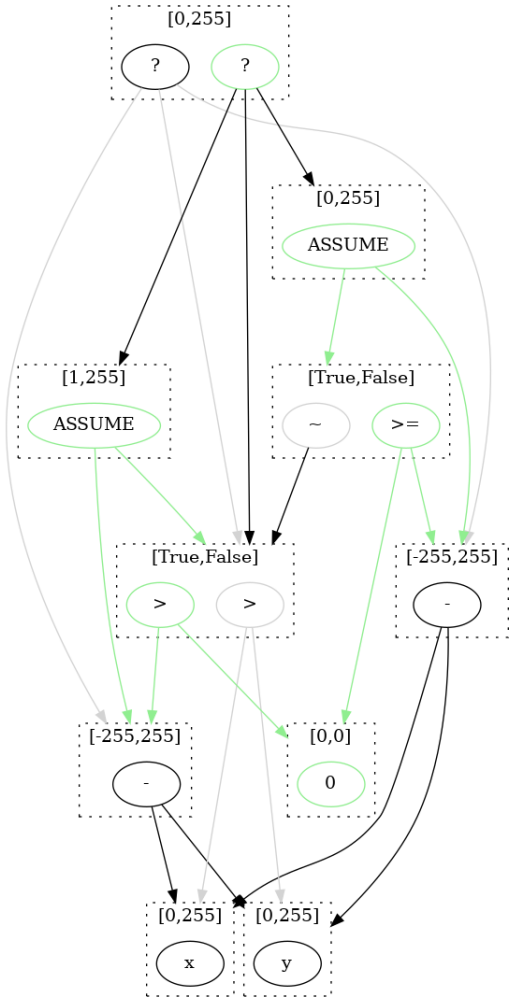


Fig. 2: An e-graph demonstrating abstract element refinement via ASSUME nodes. The original design is marked by light grey arrows. The green nodes represent the additional nodes added via rewriting that enable the interval refinement. Each e-class is associated with an interval of possible outputs.

Consider a simple example based on interval arithmetic.

$$\texttt{x>y ? x - y : y - x} \tag{12}$$

Let $\mathcal{A}[\![\texttt{x}]\!] = \mathcal{A}[\![\texttt{y}]\!] = [0, 255]$. A naive interval analysis yields:

$$\mathcal{A}[\![\texttt{x - y}]\!] = \mathcal{A}[\![\texttt{y - x}]\!] = [-255, 255] \Rightarrow$$
$$\mathcal{A}[\![\texttt{x>y ? x - y : y - x}]\!] = [-255, 255]$$

Introducing ASSUME operators as described in Section III-B yields two new expressions to interpret.

$$\mathcal{A}[\![\texttt{ASSUME(x-y, x>y)}]\!] = ? \tag{13}$$
$$\mathcal{A}[\![\texttt{ASSUME(y-x, } \sim \texttt{x>y)}]\!] = ? \tag{14}$$

Whilst the reasoning required to infer the implication of the assumed condition on the abstract element is not difficult, it is non-trivial. In contrast, by applying additional rewrites to

an e-graph containing these ASSUME operators, as shown in Figure 2, the reasoning can be simplified,

$$(13) = \mathcal{A}[\![\texttt{ASSUME}(C_1, C_1 > 0)]\!] = [-255, 255] \cap [1, \infty) = [1, 255]$$
$$(14) = \mathcal{A}[\![\texttt{ASSUME}(C_2, C_2 \geq 0)]\!] = [-255, 255] \cap [0, \infty) = [0, 255]$$

, where C_1 and C_2 denote the e-classes containing x−y and y−x, respectively. These are the two subtraction classes in Figure 2. Recall from Section II-B, that the intersection is the meet of the lattice of intervals, and that the meet of two valid abstract approximations is a valid abstract approximation. For an interval analysis, we can extend the reasoning capabilities of the ASSUME operators.

$$\mathcal{A}[\![\texttt{ASSUME}(e, C)]\!] = (\mathcal{A}[\![e]\!] \cap I)\,, \text{ where for constant } c^*$$

$$I = \begin{cases} (-\infty, c^*) & \text{if } (e < c^*) \in C \\ (-\infty, c^*] & \text{if } (e \leq c^*) \in C \\ (c^*, \infty) & \text{if } (e > c^*) \in C \\ [c^*, \infty) & \text{if } (e \geq c^*) \in C \\ [c^*, c^*] & \text{if } (e == c^*) \in C \\ (-\infty, c^*) \cup (c^*, \infty) & \text{if } (e \neq c^*) \in C \\ (-\infty, \infty) & \text{else.} \end{cases} \quad (15)$$

Propagating the abstract elements associated with ASSUME operators will yield an overall program analysis refinement. As shown in [13] and later in Section V, a tighter program analysis may enable additional rewriting opportunities. The precise abstract interpretation of ASSUME will depend upon the particular abstract domain, but the reasoning simplification due to the e-graph representation will be common to all implementations. We will demonstrate the benefits of abstraction refinement via a value range analysis implemented in our RTL optimizer in Section IV.

To perform the analysis shown in (15), we need to detect whether a particular expression is contained in the constraint e-class, for example, $(e < c^*) \in C$. This e-class C may contain compound expressions, such as $(e < c^*)\&(e > b^*)$, therefore we actually search the entire sub-e-graph. To simplify this search, we extract an analysis-friendly representative from the e-class C, which is simple to analyze. This representative extraction prioritises the expressions shown in (15). Applying standard comparator rewriting will allow the e-graph to automatically discover these analysis friendly representations.

IV. RTL Performance Optimization

In the previous section, we described theoretically how to encode constraint-aware optimization via e-graph rewriting. In this section, we demonstrate a practical realization of the theory by extending a datapath optimization tool, ROVER [3], [20], built on the egg library [2]. Figure 3 provides an overview of the tool flow. ROVER targets RTL optimization, parsing input (System) Verilog using Slang [33], and converting it to an intermediate language, VeriLang. The VeriLang language can be transformed into an e-graph by egg. ROVER's back-end generates System Verilog from a VeriLang expression, and can produce an associated formal verification proof decomposition [14], [20]. These core ROVER components were developed in prior work [3], [14], [20].

In this work, we first extend VeriLang to incorporate an ASSUME operator as described in Section III. To perform an RTL based value range analysis, we implement an e-class analysis on the VeriLang e-graph. We describe the value range analysis in detail in Section IV-A. We then develop a set of RTL rewrites that realize the constraint-aware optimizations described above. We describe these rewrites in Section IV-B. Lastly, to target performance optimization, we introduce an RTL delay model, that captures downstream logic synthesis optimizations. The cost model is described further in Section IV-C. We focus on combinational RTL designs in this work but the approach can be easily generalized to feed-forward pipelined designs [34], without considering retiming.

A. Value Range Analysis

Each signal within the design and in the corresponding e-graph represents a bitvector. We conservatively estimate the range of possible values of each signal using a finite union of integer intervals. To be precise, with each expression we associate an element of the set $\mathcal{A}$:

$$\mathcal{A} = \left\{ \bigcup_{i=1}^{n} [a_i, b_i] \,|\, a_i \leq b_i,\ a_i, b_i \in \mathbb{N},\ n \in \mathbb{N} \right\}.$$

We encode a value range analysis as an e-class analysis (as described in Section II-A), associating an element of the set $\mathcal{A}$ with each e-class. In RTL, each module input, corresponding to a variable in the e-graph, is defined with an associated bitwidth. We conservatively assume that all inputs are possible, therefore initialize the e-class analysis for an n-bit input to $[0, 2^n - 1]$. Input value constraints could be captured using an alternative initialization. To encode bitwidth truncation, we use a conservative approximation to modular intervals.

$$[l, u] \mod p = \begin{cases} [l \mod p, u \mod p] & \text{if } \left\lfloor \frac{l}{p} \right\rfloor == \left\lfloor \frac{u}{p} \right\rfloor \\ [0, p-1] & \text{else.} \end{cases}$$

We approximate the outputs of each RTL operator using an interval arithmetic that follows the Verilog semantics. We elected to store unions of unsigned integer intervals for all signals, even those defined to be signed. We encode signage in the interval analysis of the individual operators. We define two conversion functions, each parameterized by target bitwidth $n \in \mathbb{N}$, $S_n : [0, 2^n - 1]^2 \to [-2^{n-1}, 2^{n-1} - 1]^2$ mapping an unsigned interval to it's 2's complement interpretation, and $U_n : \{[a, b] \,|\, a, b \in \mathbb{Z}, a \leq b\} \to [0, 2^n - 1]^2$ mapping an arbitrary integer interval to it's unsigned interpretation.

$$S_n([x, y]) = \begin{cases} [x, y] & \text{if } y < 2^{n-1} \\ [x - 2^n, y - 2^n] & \text{elif } x \geq 2^{n-1} \\ [x, 2^{n-1} - 1] \cup [-2^{n-1}, y - 2^n] & \text{else} \end{cases}$$

$$U_n([w, z]) = \begin{cases} [0, 2^n - 1] & \text{if } \left\lfloor \frac{w}{2^n} \right\rfloor < \left\lfloor \frac{z}{2^n} \right\rfloor - 1 \\ [0, z^*] \cup [w^*, 2^n - 1] & \text{elif } \left\lfloor \frac{w}{2^n} \right\rfloor < \left\lfloor \frac{z}{2^n} \right\rfloor \\ [w^*, z^*] & \text{else} \end{cases}$$

,where $z^* = z \mod 2^n, w^* = w \mod 2^n$.

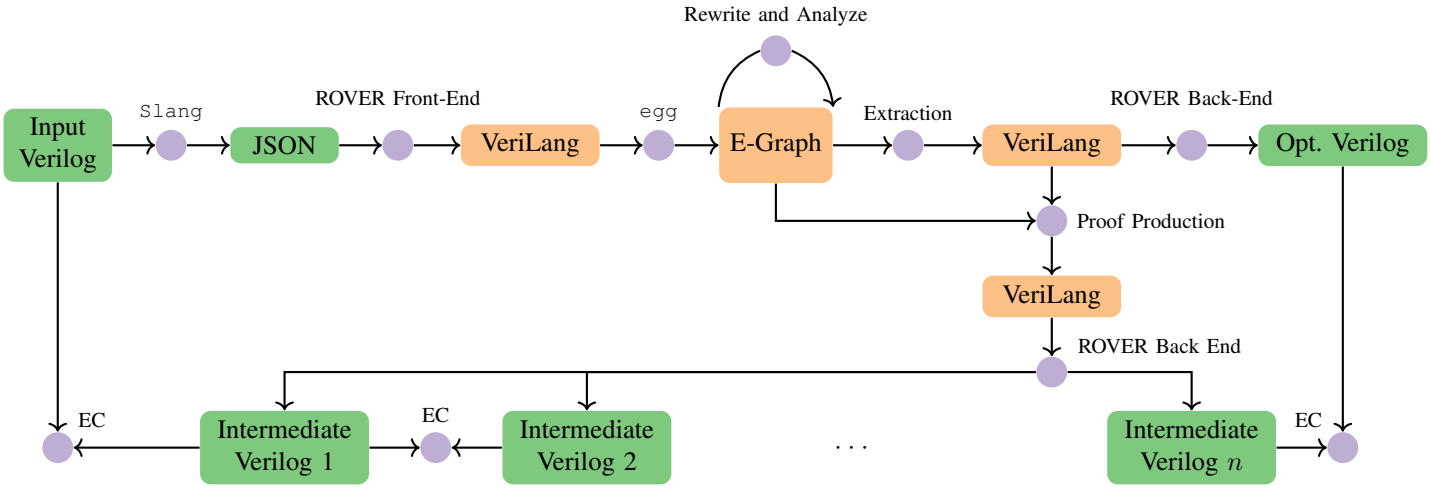


Fig. 3: Flow diagram describing the operation of ROVER. The intermediate RTL designs are formally verified to be functionally equivalent using a commercial equivalence checker (EC) forming a chain of reasoning. The orange boxes denote where our novel contributions are utilized.

These conversion functions demonstrate the need for unions of intervals, rather than simple intervals alone. We demonstrate their usage via a System Verilog example.

```
wire [7:0] a;
wire [3:0] b;
wire [5:0] add;
assign add = $signed(a[4:0]) + $signed(b);
```

Initializing $\mathcal{A}[\![\texttt{a}]\!] = [0, 255]$ and $\mathcal{A}[\![\texttt{b}]\!] = [0, 15]$, we compute

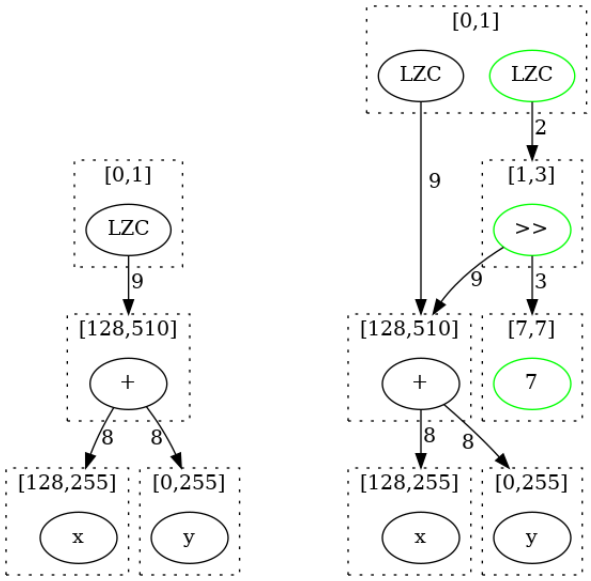
$$\begin{aligned}
&\mathcal{A}[\![\texttt{\$signed(a[4:0])} + \texttt{\$signed(b)}]\!] \\
&= S_5\left([0,255] \mod 2^5\right) + S_4\left([0,15] \mod 2^4\right) \\
&= S_5([0,31]) + S_4([0,15]) \\
&= [-16,15] + [-8,7] = [-24,22], \\
&\mathcal{A}[\![\texttt{add[5:0]}]\!] = U_6([-24,22]) = [0,22] \cup [40,63].
\end{aligned}$$

Even in this simple example, the analysis identifies that the `add` signal can only take a subset of the integer values that can be represented in six bits. Intuitively, ROVER computes the bitvector interval analysis by converting the bitvectors to integers, performing integer interval arithmetic, then converting back to bitvectors.

Since Verilog is a context determined language, the signage of an operator is determined by the signage of it's operands, as specified in the language reference manual (LRM) [35]. We use the following procedure to calculate the interval abstraction of an arbitrary operator.

1) Determine operator signage according to LRM.
2) Truncate unsigned operand intervals to operand bitwidth and convert to integer intervals according to signage.
3) Compute integer interval arithmetic abstraction of operator.
4) Convert computed interval to unsigned interpretation using the assigned to bitwidth.

To handle unions of intervals, given an interval abstraction function f, we conservatively approximate $f\left(\bigcup_i^n [a_i, b_i]\right)$ by $\bigcup_i^n f\left([a_i, b_i]\right)$. We implement a basic algorithm to reduce a given union of intervals to the minimum number of unions required. For example, $[0,3] \cup [4,7] = [0,7]$.



(a) Initial e-graph represents $\text{LZC}(x + y)$.

(b) $\text{LZC}(a) \rightarrow \text{LZC}(a \gg 7)$. Green nodes are newly added.

Fig. 4: An e-graph before and after rewriting. The LZC node denotes a leading-zero counter. Edges are labelled with bitwidths. The rewrite is valid due to the input constraint, $x \geq 128$, which implies that $\text{LZC}(x + y) \leq 1$.

The value range analysis data enables deeper rewrites that go beyond simple syntactic rewrites, such as bitwidth reduction. These rewrites will be discussed next. Furthermore when combined with the `ASSUME` node rewriting, ROVER benefits from the analysis refinement described in Section III-C.

B. Constraint and Value Range Aware RTL Rewriting

The value range analysis data can be accessed whilst rewriting, to determine the validity of a conditional rewrite. For example, let $\mathcal{A}[\![a]\!] = [a_1, a_2]$ and $\mathcal{A}[\![b]\!] = [b_1, b_2]$, then for an unsigned comparison $a > b \rightarrow \texttt{1'b1}$ is valid if $a_1 > b_2$. Similarly, the value range analysis can be used by dynamic rewrites to construct the equivalent right-hand side based on the analysis data. The most important example of this is for bitwidth reduction rewrites, where ROVER is able to shrink the bitwidth of a given operator if the value range analysis proves that the values which that operator can take can be represented in fewer bits. Consider the following Verilog.

```
fma[6:0] = a[2:0] * b[2:0] + c[2:0]
```

Value range analysis shows that $\mathcal{A}[\![\texttt{fma}]\!] = [0, 56]$, which implies that `fma` can be reduced to six bits. According to ROVER's circuit area model [3], a 6-bit adder is 19% smaller than a 7-bit adder. Logic synthesis results support ROVER's model, showing a 17% reduction in combinational cells.

In addition to bitwidth reduction, ROVER benefits from a set of hardware specific optimizations that rely on value range analysis. These are summarized in an online repository[1]. An example is shown in Figure 4, where the value range analysis proves that the result of the leading zero count (LZC) is in $[0, 1]$. This shows that the leading one must be in one of the two most significant bits and hence the LZC taking a 9-bit signal can be replaced by an LZC considering only the two most significant bits of the input.

ROVER exploits constraints present in the design itself, which are typically expressed via muxes in RTL, therefore `ASSUME` operators are introduced and propagated by applying the rewrites introduced in Table I. ROVER further benefits from hardware specific constraint-aware rewrites. For example, ROVER can detect opportunities to implement an efficient absolute difference using a compound adder [36].

$$\texttt{ASSUME}(a - b, a > b) \rightarrow \texttt{ASSUME}(\texttt{ABSDIFF}(a, b), a > b)$$

Such a rewrite is applicable in Figure 1 and in Section V-C we apply it to enable resource sharing.

ROVER's rewrites are mostly learnt from Intel engineers or are derived from efforts to decompose manual optimizations into a sequence of generally applicable transformations. In Section V, we demonstrate how constraint-aware ROVER is able automate hardware optimizations that were previously only implemented manually by skilled engineers. In total ROVER uses 179 rewrites, of which 122 are newly added.

C. Extraction

ROVER continues to apply rewrites to the e-graph until saturation (described in Section II-A) or a user defined iteration limit is reached. The final e-graph contains a set of functionally equivalent implementations. The extraction process selects a set of e-classes to implement and within these e-classes chooses the best node to implement that particular e-class. Prior work on ROVER introduced a theoretical area cost model to rank implementation candidates in terms of circuit area [3]. The theoretical model assigns an estimate of the number of two input gates required to construct a given operator as a function of input and output parameters. For logic operators the models are relatively simple, but for arithmetic operators, a fixed component architecture is selected, from which an estimate is derived. These component architectures are shown in Table II. In [20], we evaluated the fidelity of the area model.

TABLE II: VeriLang operators including the architecture used for theoretical cost assignment. Operators above the dashed line are from Verilog, whilst those below are custom operators that allow VeriLang to express more optimizations.

Operator	Symbol	Arity	Architecture
Add/Sub	+/-	2	Prefix Adder (PA)
Negation	-	1	PA [37]
Multiplication	$\times$	2	Booth Radix-4 [38]
Reduce	$\&, \|, \hat{}$	1	Log Tree
Inverse Reduce	$\sim\&, \sim\|, \sim\hat{}$	1	Log Tree
Shifting	$\ll, \gg$	2	Mux Tree
Multiplexer	$\cdot ? \cdot : \cdot$	3	Mux Gates
Concat/Repl	$\{,\}$	n	Wiring
Comparison	$==, !=$ $<, \leq$ $>, \geq$	2	PA
Register	`REG`	2	D-flip flop
Leading-Zero Count	`LZC`	1	Lookup Table
Sum	`SUM`	n	CSA and PA
Fused Mult-Add	`FMA`	3	Booth Radix-4
Absolute Difference	`ABSDIFF`	2	Compound Adder [36]

In this work, we develop a theoretical delay metric, estimating the number of two input gate delays on the critical path of each operator, as a function of the input and output parameters. Delay accumulates through combinational paths in the design, therefore the total delay to an operator's output is the maximum delay across its operands plus its own delay.

Since design delay depends only on the delay of the longest delay path, a greedy approach to extraction, selecting the minimal delay operator from each e-class, will produce the minimum delay implementation. This is implemented using the default `egg` extraction method. Such an approach does not differentiate between implementations that achieve the same delay but occupy different circuit areas.

Minimum circuit area extraction must correctly account for common sub-expression re-use and hence has been formulated as an integer linear programming (ILP) problem [3]. Let $\mathcal{N}$ denote the set of all nodes, $\mathcal{C}$ denote the set of all e-classes and $E \subseteq \mathcal{N} \times \mathcal{C}$ be the set of e-graph edges. Additionally, let $\mathcal{N}_c$ be the set of nodes in a particular e-class c. For each node $n \in \mathcal{N}$, we associate costs, area(n) and delay(n), based on the theoretical area and delay metrics, along with a binary variable $x_n \in \{0, 1\}$, indicating whether n is implemented in the final RTL. The objective function of the area minimizing ILP is described in (16). The program constraints ensure that we extract a valid circuit description. The first constraint (17) ensures that at least one node from all children e-classes of a selected node is implemented. The final constraint (18) ensures that for all output expressions found in the set of e-classes $\mathcal{S}$, we generate a circuit producing that output.

[1]https://figshare.com/s/e3ab2850662d24991cbc

$$\text{minimize: } \sum_{n \in \mathcal{N}} \text{area}(n) x_n \text{ subject to:} \tag{16}$$

$$\forall (n, c) \in E.\ x_n \leq \sum_{n' \in \mathcal{N}_c} x_{n'} \tag{17}$$

$$\forall c \in \mathcal{S}.\ \sum_{n \in \mathcal{N}_c} x_n = 1. \tag{18}$$

Since e-graphs may contain cycles we include additional topological sorting variables associated with each class t_c. Let N denote the number of e-classes and $\mathcal{C}(n)$ be the e-class containing node n. The constraint (19) ensures that the output expression is acyclic.

$$\forall (n, k) \in E \quad t_{\mathcal{C}(n)} - N x_n - t_k \geq 1 - N \tag{19}$$

Selecting a node $n \in \mathcal{N}_c$ with child k, *i.e.* $x_n = 1$, the constraint simplifies to $t_c \geq t_k + 1$ to get a topologically sorted result. For $x_n = 0$, the constraint is vacuously satisfied.

In this work, we extend this ILP encoding. We associate a new integer variable, d_c, with each e-class $c \in \mathcal{C}$. We also define a new set $\mathcal{L} \subseteq \mathcal{N}$, of leaf nodes with no children and for each $n \in \mathcal{L}$ we associate a, potentially zero, input delay input_delay(n). To correctly model pipelined circuits, define $E_R \subseteq E$ of edges originating from register operators. To meet a given delay target $d > 0$, we add the following constraints.

$$\forall (n, c) \in E \setminus E_R.\ d_{\mathcal{C}(n)} \geq d_c + x_n \times (\text{delay}(n) + K) - K \tag{20}$$

$$\forall n \in \mathcal{L}.\ d_{\mathcal{C}(n)} \geq \text{input_delay}(n) \tag{21}$$

$$\forall c \in \mathcal{C}.\ d_c \leq d \tag{22}$$

Constraint (20) ensures that delay accumulates throughout combinational paths in the design and (21) encodes the input delays. The large arbitrary constant K ensures that (20) is vacuously satisfied if $x_n = 0$. Lastly, (22) ensures that all paths in the extracted circuit meet the delay target d. ROVER deploys the CBC solver [39] yielding the minimum area implementation that meets the total delay constraint d.

Using the greedy extraction method described earlier, ROVER obtains d_{min}, the minimum delay that any design in the e-graph can meet. The minimum area implementation in the e-graph is obtained by solving the original ILP problem, omitting the new delay constraints (20), (21) and (22). This minimum area design has corresponding delay d_{max}. Note that, the area model does not differentiate between designs with the same area cost but different performance. Solving an ILP problem for each delay target $d \in [d_{min}, d_{max})$ yields a complete Pareto frontier of implementations minimizing circuit area at each delay target. ROVER terminates it's delay sweep from d_{min} to d_{max} if, for some $d < d_{max}$, the extracted circuit matches the minimum area circuit obtained at d_{max}.

The extraction process generates a set of VeriLang expressions, from which ROVER's back-end generates a set of System Verilog implementations. For each generated implementation, ROVER produces an accompanying sequence of intermediate Verilog designs that can be used to verify that each implementation matches the input design, as shown in Figure 3. ROVER's back-end and verification method have been described in prior work [3], [14].

V. Results

To demonstrate the value of constraint-aware optimization, we first provide an in-depth case-study, analyzing exactly how ROVER optimized a floating-point component. Next, we use a second case-study, a component from a floating-point dot product, to analyze the results of the Pareto frontier extraction method, described in Section IV-C. Finally, to show its applicability more generally we use constraint-aware ROVER to optimize a range of benchmarks.

To evaluate the quality of the ROVER's optimizations, we synthesize the baseline and ROVER generated designs using a commercial synthesis tool for a TSMC 5nm cell library. The commercial logic synthesis tool includes state-of-the-art datapath optimizations by default [27], [40]. We use the commercial tool, optimizing the baseline design as a comparison point throughout. The functional equivalence of the original and ROVER generated designs is formally verified using the approach described in [14].

A. Case-Study: Floating-Point Subtract

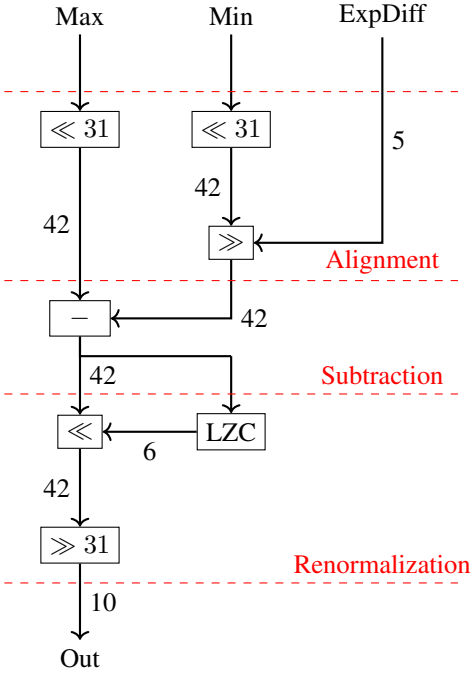
To illustrate the new optimization capabilities in ROVER, we take a complex datapath block, a floating-point subtractor computing $2^{ea} \times 1.ma - 2^{eb} \times 1.mb$ and producing a floating-point output. The subtraction case raises the potential for cancellation and hence is more complex than the pure addition case [8]. The architecture shown in Figure 5a is the core datapath of a baseline half-precision (FP16) floating-point subtractor computing the output mantissa. The core datapath takes sorted mantissas Max and Min, along with an ExpDiff$= |ea - eb|$. This architecture is easy to write and simple to verify. It is comprised of three main components: alignment, subtraction and normalization. The baseline architecture essentially converts the input mantissas to a 42-bit aligned integer representation, performs a 42-bit subtraction, then normalizes using an LZC, truncating the resulting signal.

Floating-point arithmetic operators have been well-studied in academia and industry, providing optimized single-path and dual-path implementations. The dual-path architecture, also known as the near-path/far-path optimization, is derived from the observation that no input fully exercises the critical path of the baseline architecture [8]. The dual-path architecture inserts a case-split using a positive constant c, taking the near-path for small ExpDiff values (ExpDiff $\leq c$) and an alternative far-path for large ExpDiff values (ExpDiff $> c$). On the near-path, close floating point values are subtracted yielding a small alignment shift. On the far-path, catastrophic cancellation is avoided, reducing the renormalization logic.

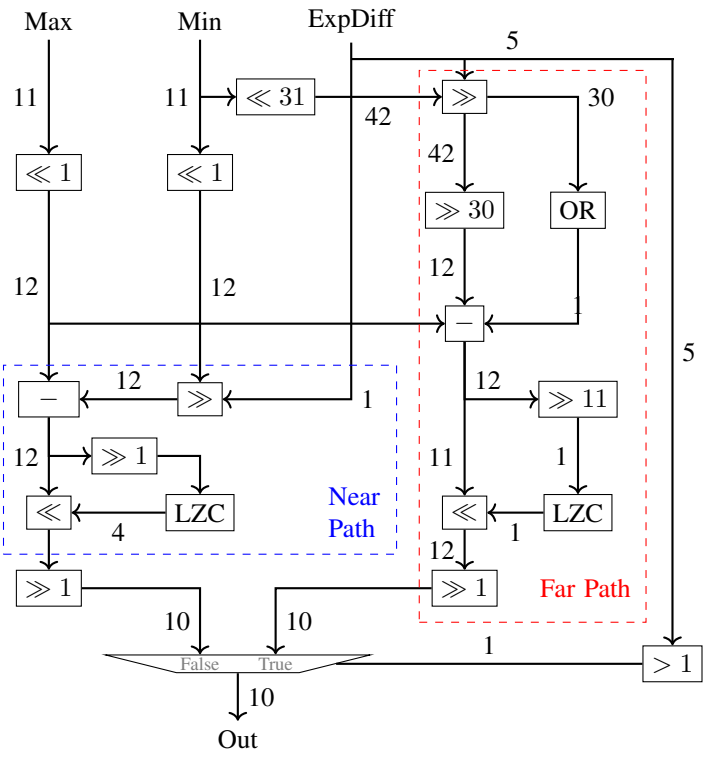
To explore alternative case-splitting constants c, we manually insert a mux on the output of the baseline architecture (Figure 5a).

$$\text{ExpDiff} > c \ ?\ \text{Out} \ :\ \text{Out} \tag{23}$$

Such a redundant case-split would be optimized away by most tools, but with ROVER's constraint-aware optimizations,



(a) Baseline architecture, using a 42-bit subtraction.



(b) Dual-path optimized architecture, using two 12-bit subtractors. Near-path uses a one bit alignment shift, followed by a larger renormalization stage. Far-path uses a 5-bit alignment shift and a single bit renormalization.

Fig. 5: Half-precision floating-point subtractor architectures. Input mantissas have the implicit one appended. Edge labels represent bitwidths. The floating-point inputs are sorted, such that Max $\geq$ (Min$\gg$ExpDiff). In the optimization, we omitted input sorting and exponent difference calculation blocks to focus on the core datapath.

ASSUME operators are propagated down each branch, triggering a chain of branch specific optimizations that result in efficient dual-path implementations. In Figure 6, we use ROVER to optimize dual-path architectures for each value of $c \in \{1, 2, 4, 8\}$. ROVER's cost-model suggests that the optimal split uses $c = 1$, in agreement with computer arithmetic literature [8]. In Figure 5b, we present the ROVER generated dual-path architecture for $c = 1$. The design uses two small 12-bit subtractors, replacing the larger 42-bit subtractor in the baseline architecture. To understand how ROVER generated the dual-path design, we now describe the rewrites ROVER applied to transform Figure 5a into Figure 5b.

On the near-path ROVER constrains the ExpDiff ≤ 1, allowing the initial alignment shift to be shrunk, using the following rewrite (with $i = 1$) that exploits the domain refinement described in Section III-C.

$$y \in [0, 2^i - 1] \Rightarrow x \gg y \rightarrow x \gg y[i-1:0] \qquad (24)$$

Having reduced the shift, ROVER can then apply a rewrite to re-order shift operations:

$$\texttt{max}(y) < 31 \Rightarrow (x \ll 31) \gg y \rightarrow ((x \ll \texttt{max}(y)) \gg y) \ll (31 - \texttt{max}(y))$$

, where $\texttt{max}(y)$ returns the maximum value of the signal y according to ROVER's value range analysis. These rewrites lead ROVER to discover the following sub-expression, that yields a clear opportunity to reduce a 42-bit subtraction to a 12-bit subtraction:

$$(\text{Max} \ll 31) - ((\text{Min} \ll 1 \gg \text{AExpDiff}) \ll 30) \rightarrow (\text{Max} \ll 1 - (\text{Min} \ll 1 \gg \text{AExpDiff})) \ll 30$$

, where $\text{AExpDiff} = \texttt{ASSUME}(\text{ExpDiff}, \text{ExpDiff} \leq 1)$.

This shifted subtraction is passed into a normalization circuit (including an LZC). The constant shift and normalization order are swapped by ROVER, reducing the critical path.

$$(x \ll 30) \ll LZC(x \ll 30) \rightarrow (x \ll LZC(x)) \ll 30 \qquad (25)$$

This sequence of rewrites has essentially pulled the initial alignment shifts through the design until they reach the output, where they meet the truncation shift. With a final rewrite these constant shifts cancel out, resulting in the final shift by one shown in Figure 5b.

On the far-path, via ASSUME node propagation, ROVER constrains ExpDiff> 1. This fact is propagated through the e-graph via the value range analysis, which leads to the automated discovery that the leading zero count on this branch is less than two, since cancellation due to subtraction is limited. This discovery allows ROVER to reduce the 42-bit LZC to an

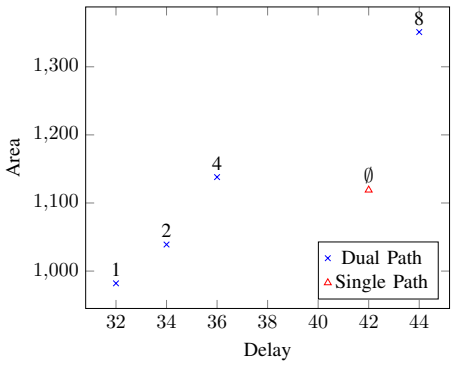


Fig. 6: ROVER optimized designs for different case-split values, the label above each point represents the value on which the corresponding design case-splits. The area and delay values are those calculated by ROVER's cost models.

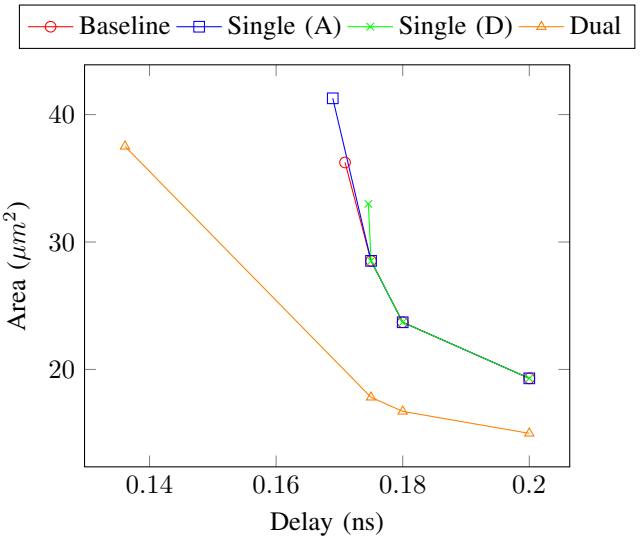


Fig. 7: Area-delay profiles for four FP16 Subtractor implementations. Single (A/D) are ROVER's single-path designs optimizing for area or delay, respectively. Dual represents the ROVER generated design shown in Figure 5b.

LZC on the most-significant bit only. Simultaneously, ROVER deploys another shift transformation.

$$(x \ll y) \gg 31 \rightarrow ((x \gg 31 - \max(y)) \ll y) \gg \max(y)$$

In this example, y is the LZC output, therefore $\max(y) = 1$. These transformations leave ROVER with the following expression in the e-graph:

$$(x - y) \gg 30 \rightarrow (x \gg 30) - (y \gg 30) - (x_{30} < y_{30}) \quad (26)$$

, where we use x_{30} and y_{30} to denote truncation to the least-significant 30 bits. In the floating-point subtraction context, the final term of the subtraction can be further reduced by ROVER to an or reduction since $x_{30} = (\text{Max} \ll 31)_{30}$, which is zero. Whilst we have shown rewrites including specific constant values, ROVER's rewrites dynamically adapt to arbitrary constant values.

The transformations applied to each branch by ROVER substantially reduce the critical path delay, leading to the dual-path architecture shown in Figure 5b. The ROVER generated architecture re-discovers optimizations typically implemented by hand by experienced hardware engineers. As mentioned above, ROVER allowed us to automatically explore alternative case-splitting constants which we synthesize in Figure 6. ROVER can also optimize the single-path architecture, shown in Figure 5a, directly, generating the "Single-Path" implementation plotted in Figure 6. To achieve this result, we added a rewrite purely to aid the value range analysis.

$$x - (y \gg z) \rightarrow z > 0 \; ? \; x - (y \gg z) : x - y \quad (27)$$

This transformation treats $z = 0$ as a special case, helping ROVER's value range analysis to recognize some of the optimization opportunities that were utilized in the dual-path case. As we can see in Figure 6, the dual-path architectures are considered superior by ROVER for constants $c \leq 4$.

The dual-path architecture is generated by ROVER in less than 10 seconds, growing from an initial 100 node e-graph to an e-graph containing over 400 nodes. The majority of the runtime is spent in the rewrite application and analysis phases. In Figure 7, we plot the area-delay curves generated from synthesizing four different architectures. The baseline, two ROVER generated single-path designs optimizing for minimal area and minimal delay, respectively, and one ROVER generated dual-path design (Figure 5b). The results clearly show that the ROVER generated dual-path architecture is superior, offering a 20% performance improvement for a 3% area penalty, when compared against the baseline architecture optimized by logic synthesis. In contrast, despite exhibiting significant architectural differences, the performance of the two single-path designs and baseline design are very close. This implies that the commercial synthesis tool is able to express most of the optimizations present in the ROVER generated single-path designs. However, the commercial synthesis tool does not appear to be capable of constraint-aware optimizations, the key component required to reproduce the dual-path architecture.

B. Multi-Objective Optimization

In Section IV-C, we described an algorithm to extract a Pareto frontier of implementations, balancing the area-delay trade-off. We demonstrate the value of such an approach via another case-study, a $2K$-input floating-point dot product.

$$2^{ea_1} ma_1 \times 2^{eb_1} mb_1 + \cdots + 2^{ea_K} ma_K \times 2^{eb_K} mb_K =$$
$$2^{ea_1+eb_1}(ma_1 \times mb_1) + \cdots + 2^{ea_K+eb_K}(ma_K \times mb_K)$$

The hardware implementation of this equation computes each product, producing K product exponents $(ea_i + eb_i)$ and associated product mantissas $(ma_i \times mb_i)$. To perform the final accumulation, the hardware must determine the maximum exponent and compute an alignment factor for each product. This corresponds to a maximum comparison tree followed by K subtractions, as shown in Figure 8.

Feeding the baseline implementation into ROVER, it manipulates the mux tree structure, introducing additional subtraction and comparison operators, increasing the degree of

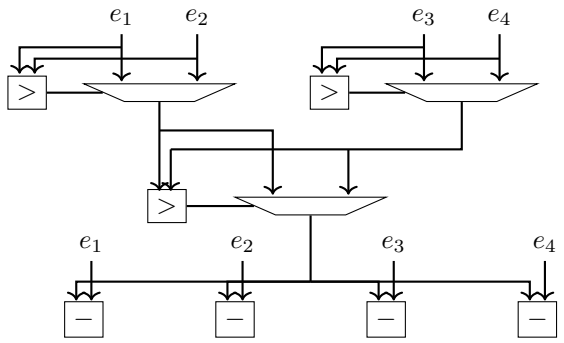


Fig. 8: Component from a floating-point dot product design, calculating the alignment factor for each product. The hardware computes, for $i = 1, 2, 3, 4$, $\max_j e_j - e_i$.

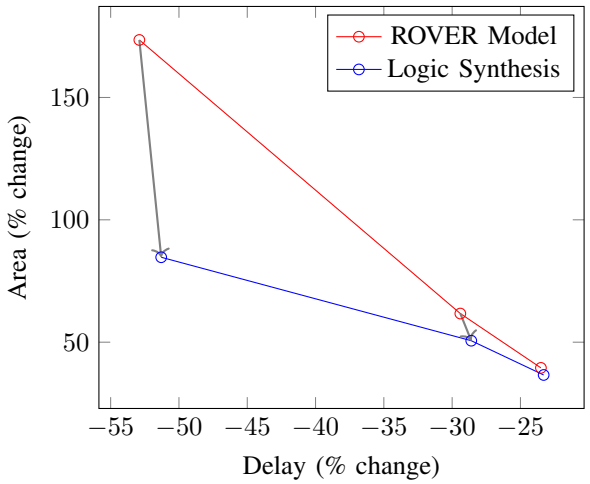


Fig. 9: Pareto frontier of design costs from a 4-input maximum comparison tree based on ROVER's theoretical cost models and logic synthesis. We plot the percentage change comparing against the baseline implementation. Grey arrows connect results derived from the same implementation.

speculation. ROVER's e-graph explores all degrees of parallelism available, leaving the extraction phase to determine the optimal implementation based on the delay target. We omit the `ASSUME` node rewrites here as the large number of correlated muxes in the design leads to excessive e-graph duplication. ROVER generates a 202 node e-graph during exploration, from which multiple implementations are selected.

Given the max tree in Figure 8, ROVER generates 18 distinct delay constrained ILP problems, which take 8 seconds to solve in total. These solutions yield a Pareto frontier containing four distinct implementations. The minimum area architecture returned is unchanged from the input shown in Figure 8. The estimates given by ROVER's cost model for each implementation are shown in Figure 9, using the input architecture as the baseline. The synthesis results closely match the ROVER predicted trend, with one exception. For the implementation with the highest performance, ROVER over-predicts the area penalty, since ROVER's area model fails to capture all the sharing opportunities exploited by logic synthesis.

C. Delay Optimization Evaluation

Having analyzed two case-studies, we now demonstrate that constraint-aware optimization is generally applicable. In Table III, we report synthesis results for a range of benchmarks, where we used ROVER to optimize the designs. For a baseline, we use the commercial logic synthesis tool with all datapath optimizations enabled, including datapath clustering, constant folding and common sub-expression sharing [27]. To highlight the benefits of both the value range analysis and the constraint-aware optimizations, we conduct an ablation study and compare versions of ROVER with and without these features. Across the entire benchmark set, constraint-aware ROVER with value range analysis reduces critical path delay by 30% on average combined with a 1% average circuit area reduction, when compared to the baseline. We omit total circuit power, but this is generally closely correlated with circuit area, seeing an average 1% increase. The ablation study demonstrates that, when compared against prior work on ROVER [20], the value range analysis alone reduces delay by 3% on average, whilst adding constraint-aware optimizations leads to a 20% average reduction. To understand the resulting improvement, we now discuss each benchmark in detail.

The "Media Kernel" is a kernel from the Intel media module, computing an interpolation between four pixels then clamping the result. For particular clamping thresholds, ROVER's value range analysis proves that the threshold can never be breached, allowing ROVER to remove the clamping. This example demonstrates the advantage of combining rewriting with value range analysis. Before applying rewrites, ROVER's value range analysis fails to prove that the threshold cannot be breached. After several rewriting iterations, the value range analysis is refined sufficiently to prove the desired property, triggering the removal of the clamping. This optimization only relies on the value range analysis, so constraint-aware ROVER produces an identical implementation.

The "Unorm8 to FP32" benchmark converts a value in a Unorm8 format to an FP32 format, implementing round towards zero. The baseline implementation handles zero inputs on a separate path. Constraint-aware ROVER exploits this to exclude zero from the input domain of the alternative path, leading to a set of constraint-aware optimizations that reduce the critical path. The "FP16 to Unorm11" benchmark performs a reverse conversion, taking an input in FP16 format, and converting it to a Unorm11 format. ROVER re-uses several of the subtraction and rounding rewrites introduced in Section V-A. A lack of data dependent muxes means that constraint-aware optimization offers no further benefit.

We then include the results for both 8-bit and 16-bit floating-point subtractors, based on the architecture described in Section V-A. Both examples benefit from the constraint-aware optimizations, with the FP8 benchmark showing a slightly smaller advantage from the dual-path implementation. In both examples, circuit power consumption increases since the dual-path architecture executes two computational paths in parallel. The "Normalization" benchmark is a kernel from the Intel elementary function unit, normalizing a floating-point value. As in the "FP Sub" benchmarks, a manually inserted case-split

TABLE III: Logic synthesis results under a zero delay synthesis constraint. We compare the baseline, ROVER [20], ROVER with only value range analysis (+ VRA) and Constraint-Aware ROVER with value range analysis (+ VRA). Delay and area are measured in ps and μm^2, respectively. We bold the best result for each metric and report percentage improvement against the baseline. † denotes that we ran ROVER in two passes.

Benchmark	E-Graph		Baseline		ROVER [20]		ROVER + VRA		Constraint-Aware ROVER + VRA			
	Initial	Final	Delay	Area	Delay	Area	Delay	Area	Delay		Area	
Media Kernel	40	645	288.6	192.2	267.5	199.2	**241.4**	**156.7**	**241.4**	-16.3%	**156.7**	-18.5%
Unorm8 to FP32	32	88	36.7	11.2	27.0	9.1	33.9	**8.0**	**23.2**	-36.8%	8.2	-26.5%
FP16 to Unorm11	44	71	54.3	12.8	54.5	15.0	**31.5**	**3.0**	**31.5**	-41.9%	**3.0**	-76.6%
FP16 Sub	79	416	170.9	36.2	176.5	**34.7**	180.9	37.7	**136.1**	-20.3%	37.5	+3.5%
FP8 Sub	86	405	68.9	13.6	68.9	13.6	68.6	**11.7**	**58.3**	-15.4%	14.1	+3.8%
Normalization	134	493	173.5	54.0	167.6	**52.1**	168.6	**52.1**	**135.2**	-22.0%	77.6	+43.5%
Max Tree (†)	43	960	173.7	**33.9**	89.8	51.2	89.8	51.2	**78.1**	-55.0%	56.4	+66.5%

is automatically optimized by ROVER, producing an efficient dual-path implementation.

The "Max Tree" was introduced in Section V-B. Due to the large number of muxes in the design, ASSUME node rewrites excessively duplicate the e-graph. To combat this we apply ROVER in two passes. First ROVER establishes an efficient mux structure, and then, by enabling the ASSUME nodes on a second pass, ROVER discovers resource sharing opportunities via the absolute difference rewrite introduced in Section IV-B.

In Section III-A, we described a live-class analysis and claimed that it limited the propagation of ASSUME operators. To evaluate this, we apply only the rewrites in Table I to the "FP16 to Unorm11" benchmark. Using the live-class analysis to conditionally create and propagate ASSUME operators, 48% fewer nodes are added to the e-graph at saturation.

Across the benchmark set, ROVER ran for an average of 69 seconds, whilst the commercial logic ran for between one and two minutes in each cases. The ROVER generated e-graphs were 11 times larger than the initial e-graph, on average. The input benchmarks contained an average 65 lines of System Verilog. For several benchmarks the extraction phase dominated the runtime, hitting the 120 second ILP timeout limit. Efficient e-graph extraction is the subject of a broader community effort to which we actively contribute[2].

VI. Conclusion

In this paper, we have presented a theoretical framework for expressing constraint-aware optimizations in an e-graph. We achieve this by introducing the ASSUME operator, for which we provide a complete semantics. This theoretical description allows e-graph users to exploit context that can be derived directly from the programs they wish to optimize. Crucially, we have described an approach to express sub-domain equivalences and a way to take advantage of the natural program analysis refinements generated by the ASSUME nodes. To demonstrate real-world value, we applied the theoretical contributions to ROVER, an RTL optimization tool. Through a floating-point subtraction case-study, we demonstrated how constraint-aware optimization can be leveraged to re-discover efficient designs, previously only implemented manually by expert designers. A second case-study highlighted how we can use a single e-graph to explore the area-delay trade-offs. Benchmark results provide evidence of general applicability.

This paper provides one approach to encoding context within the e-graph, but there have recently been new methods proposed [41] to resolve this challenge. We hope to contribute to these broader discussions and will provide the benchmarks described here as a way to compare the different methods. To ROVER, we will incorporate new ways to infer context from the input RTL. For example, in System Verilog, designers are encouraged to encode design constraints using assertions in their code. In the future, ROVER will convert these asserts directly to ASSUME operators, providing an interface for engineers to express design constraints. Finally, in this work, we manually inserted a known valuable case-split, but to achieve full automation, we aim to automatically synthesize profitable case-splits by detecting under-utilization of the critical path. The theoretical cost model is currently limited to fixed component-level architectures, however future work will incorporate architecture selection into the optimization.

[2]https://github.com/egraphs-good/extraction-gym

Acknowledgment

Thank you to Pavel Panchekha and Max Willsey for their suggestions to use ASSUME nodes to enable these techniques in the existing e-graph data structure. Thank you to Gus Henry-Smith for his suggestion to use a live-variable analysis.

References

[1] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner, "Equality saturation: A new approach to optimization," in *ACM SIGPLAN Notices*, vol. 44, no. 1. Association for Computing Machinery, 2009.

[2] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "Egg: Fast and extensible equality saturation," in *Proceedings of the ACM on Principles of Programming Languages*, vol. 5, no. POPL, 2021.

[3] S. Coward, G. A. Constantinides, and T. Drane, "Automatic Datapath Optimization using E-Graphs," in *IEEE 29th Symposium on Computer Arithmetic (ARITH)*. IEEE, 9 2022, pp. 43–50.

[4] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, "IMpress: Large Integer Multiplication Expression Rewriting for FPGA HLS," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–10.

[5] C. Nandi, M. Willsey, A. Zhu, Y. R. Wang, B. Saiki, A. Anderson, A. Schulz, D. Grossman, and Z. Tatlock, "Rewrite rule inference using equality saturation," in *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, 2021.

[6] P. Ferrara, L. Negrini, V. Arceri, and A. Cortesi, "Static analysis for dummies: Experiencing LiSA," in *SOAP 2021 - Proceedings of the 10th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis, co-located with PLDI 2021*, 2021.

[7] J. Sohn and E. E. Swartzlander, "Improved architectures for a fused floating-point add-subtract unit," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 59, no. 10, 2012.

[8] A. Beaumont-Smith, N. Burgess, S. Lefrere, and C. C. Lim, "Reduced latency IEEE floating-point standard adder architectures," *Proceedings - Symposium on Computer Arithmetic*, 1999.

[9] S. Coward, G. Constantinides, and T. Drane, "Automating Constraint-Aware Datapath Optimization using E-Graphs," in *Design Automation Conference*, 2023.

[10] C. G. Nelson, "Techniques for program verification," Ph.D. dissertation, Stanford University, 1980.

[11] L. De Moura and N. Bjørner, "Efficient E-matching for SMT solvers," in *Automated Deduction–CADE-21: 21st International Conference on Automated Deduction Bremen, Germany, July 17-20, 2007 Proceedings 21*, vol. 4603 LNAI, 2007.

[12] D. Detlefs, G. Nelson, and J. B. Saxe, "Simplify: A theorem prover for program checking," *Journal of the ACM*, vol. 52, no. 3, 2005.

[13] S. Coward, G. A. Constantinides, and T. Drane, "Combining E-Graphs with Abstract Interpretation," in *SOAP 2023 - Proceedings of the 12th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis, Co-located with PLDI 2023*, 2023.

[14] O. Flatt, S. Coward, M. Willsey, Z. Tatlock, and P. Panchekha, "Small Proofs from Congruence Closure," in *Formal Methods in Computer-Aided Design*, 9 2022.

[15] L. De Moura and N. Bjørner, "Z3: An efficient SMT Solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, vol. 4963 LNCS. Springer, 2008.

[16] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, "Automatically improving accuracy for floating point expressions," *ACM SIGPLAN Notices*, vol. 50, no. 6, pp. 1–11, 2015.

[17] E. Ustun, C. Yu, and Z. Zhang, "Equality Saturation for Datapath Synthesis: A Pathway to Pareto Optimality," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023.

[18] A. Wanna, S. Coward, T. Drane, G. A. Constantinides, and M. D. Ercegovac, "Multiplier Optimization via E-Graph Rewriting," in *Asilomar Conference on Signals, Systems and Computers*, 12 2023.

[19] J. Cheng, S. Coward, L. Chelini, R. Barbalho, and T. Drane, "SEER: Super-Optimization Explorer for HLS using E-graph Rewriting with MLIR," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 8 2023. [Online]. Available: https://doi.org/10.1145/3620665.3640392

[20] S. Coward, T. Drane, and G. A. Constantinides, "ROVER: RTL Optimization via Verified E-Graph Rewriting," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, p. 1, 2024.

[21] E. Singher and S. Itzhaky, "Easter Egg: Equality Reasoning Based on E-Graphs with Multiple Assumptions," in *Formal Methods in Computer-Aided Design*, 2024, p. 70.

[22] K. D. Cooper and L. Torczon, *Engineering a compiler: Second edition*. Elsevier, 2011.

[23] P. Cousot, *Principles of Abstract Interpretation*, 1st ed. MIT Press, 9 2021.

[24] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Conference Record of the Annual ACM Symposium on Principles of Programming Languages*, vol. Part F130756. Association for Computing Machinery, 1977.

[25] M. Daumas and G. Melquiond, "Certification of bounds on expressions involving rounded operators," *ACM Transactions on Mathematical Software*, vol. 37, no. 1, 2010.

[26] P. Talbot, D. Cachera, E. Monfroy, and C. Truchet, "Combining constraint languages via abstract interpretation," in *Proceedings - International Conference on Tools with Artificial Intelligence, ICTAI*, vol. 2019-November. IEEE, 2019.

[27] R. Zimmermann, "Datapath synthesis for standard-cell design," in *19th IEEE Symposium on Computer Arithmetic*, 2009.

[28] Synopsys, "Coding Guidelines for Datapath Synthesis," Synopsys, Mountain View, Tech. Rep., 12 2019.

[29] Siemens Digital Industries Software, "Automatic sequential clock gating with PowerPro," 2021.

[30] A. K. Verma, P. Brisk, and P. Ienne, "Data-flow transformations to maximize the use of carry-save representation in arithmetic circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 10, pp. 1761–1774, 2008.

[31] C. Seger, "Voss II," Chalmers, 2023. [Online]. Available: https://github.com/TeamVoss/VossII

[32] J. Pope and C.-J. H. Seger, "Bifröst: Creating Hardware With Building Blocks," in *2023 Forum on Specification & Design Languages (FDL)*, 2023, pp. 1–8.

[33] M. Popoloski, "Slang," 2023. [Online]. Available: https://github.com/MikePopoloski/slang

[34] S. Coward, T. Drane, E. Morini, and G. A. Constantinides, "Combining Power and Arithmetic Optimization via Datapath Rewriting," in *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. IEEE, 6 2024, pp. 24–31.

[35] D. Thomas and P. Moorby, *The Verilog® hardware description language*. Springer Science & Business Media, 2008.

[36] F. de Dinechin and M. Kumm, *Application-Specific Arithmetic*. Cham: Springer International Publishing, 2024.

[37] A. Beaumont-Smith and C.-C. Lim, "Parallel prefix adder design," in *Proceedings - IEEE Symposium on Computer Arithmetic*, 2001, pp. 218–225.

[38] I. Koren, *Computer arithmetic algorithms*. AK Peters/CRC Press, 2018.

[39] J. Forrest, T. Ralphs, H. G. Santos, S. Vigerske, J. Forrest, L. Hafer, B. Kristjansson, jpfasano, EdwinStraver, M. Lubin, Jan-Willem, rlougee, jpgoncal1, S. Brito, h-i-gassmann, Cristina, M. Saltzman, tosttost, B. Pitrus, F. MATSUSHIMA, and to-st, "coin-or/Cbc: Release releases/2.10.10," 4 2023. [Online]. Available: https://zenodo.org/record/7843975

[40] R. Zimmermann and D. Q. Tran, "Optimized synthesis of sum-of-products," in *Conference Record of the Asilomar Conference on Signals, Systems and Computers*, vol. 1, 2003.

[41] T. Hou, S. Laddad, and J. M. Hellerstein, "Towards Relational Contextual Equality Saturation," 2024.



Samuel Coward received a BSc in Mathematics in 2018 and an MPhil in Scientific Computing in 2019, from the University of Cambridge, UK. He is currently studying for a PhD in Electrical and Electronic Engineering at Imperial College London, whilst also working as a Graphics Hardware Engineer at Intel Corporation. Samuel's research focuses on automating RTL design and program analysis techniques to increase chip design productivity and quality.



George A. Constantinides (S'96, M'01, SM'08) received the Ph.D. degree from Imperial College London in 2001. Since 2002, he has been with the faculty at Imperial College London, where he is currently Professor of Digital Computation and Associate Dean of Engineering. He has served as chair of the FPGA, FPL and FPT conferences. He currently serves on several program committees and has published over 200 research papers in peer refereed journals and international conferences. Prof. Constantinides is a Senior Member of the IEEE and a Fellow of the British Computer Society.



Theo Drane started working for the Datapath consultancy, Arithmatica, in 2002 after a Mathematics degree from the University of Cambridge, UK. He moved to Imagination Technologies in 2005, where he subsequently founded their Datapath team while studying for a PhD at Imperial College London's EEE Department. In December 2018, after a stint within Cadence Design System's Logic Synthesis division, Genus, he joined Intel's Accelerated Computing Systems and Graphics Group. His applied research Numerical Hardware & System Level Design Group focuses on architecting, implementing, optimizing and formally verifying math intensive hardware.